Fuzzy System Core Functions

Code Listing

```python
import numpy as np

def gaussian_weight(x, x_center, sd):
    return np.exp(-((x - x_center) / sd) ** 2)

def rules_numerator(y_rule, z_rule):
    return np.sum(y_rule * z_rule)

def weights_denominator(z_rule):
    return np.sum(z_rule)

def normalized_output(numer, denom):
    return numer / denom

def half_mse(f, y):
    return ((f - y) ** 2) / 2

def update_consequent(y_rule, lr, f, y, denom, z_rule):
    return y_rule - lr * (f - y) * z_rule / denom

def update_center(x_center, lr, f, y, denom, z_rule, y_rule, x, sd):
    return x_center - lr * ((f - y) / denom) * (y_rule - f) * z_rule * 2 *
        (x - x_center) / (sd ** 2)

def update_spread(sd, lr, f, y, denom, z_rule, y_rule, x, x_center):
    return sd - lr * ((f - y) / denom) * (y_rule - f) * z_rule * 2 * ((x -
        x_center) ** 2) / (sd ** 3)

def true_g(u):
    return 0.6 * np.sin(np.pi * u) + 0.3 * np.sin(np.pi * 3 * u) + 0.1 *
       np.sin(np.pi * 5 * u)
```

Listing 1: Core functions for Gaussian-based fuzzy system.

Explanation

This section defines the core mathematical functions needed to implement a Gaussian-based fuzzy inference system with parameter learning.

1. `gaussian_weight(x, x_center, sd)` Computes the degree of membership using a Gaussian function:

$$\mu(x) = \exp\left(-\left(\frac{x-c}{\sigma}\right)^2\right)$$

where c is the membership function center and σ is the spread.

2. `rules_numerator(y_rule, z_rule)` Calculates the numerator of the fuzzy output aggregation formula:

$$\text{numerator} = \sum_i y_i \cdot z_i$$

Here, y_i is the consequent value of the i-th rule, and z_i is its firing strength.

3. `weights_denominator(z_rule)` Computes the denominator of the output formula:

$$\text{denominator} = \sum_i z_i$$

This ensures normalization over all rule strengths.

4. `normalized_output(numer, denom)` Computes the normalized fuzzy system output:

$$f(x) = \frac{\text{numerator}}{\text{denominator}}$$

5. `half_mse(f, y)` Computes the half Mean Squared Error between predicted output f and target y:

$$E = \frac{(f-y)^2}{2}$$

This is a common choice for gradient-based learning.

6. `update_consequent(...)` Updates the rule consequent parameters y_i using gradient descent:

$$y_i \leftarrow y_i - \eta \frac{(f-y) \cdot z_i}{\text{denom}}$$

where η is the learning rate.

7. `update_center(...)` Updates the Gaussian center c_i of each membership function:

$$c_i \leftarrow c_i - \eta \frac{(f-y)}{\text{denom}} (y_i - f) \cdot z_i \cdot \frac{2(x-c_i)}{\sigma_i^2}$$

This shifts the membership function center to reduce the prediction error.

8. `update_spread(...)` Updates the Gaussian spread σ_i of each membership function:

$$\sigma_i \leftarrow \sigma_i - \eta \frac{(f-y)}{\text{denom}} (y_i - f) \cdot z_i \cdot \frac{2(x-c_i)^2}{\sigma_i^3}$$

This adjusts the width of the Gaussian to optimize coverage.

9. `true_g(u)` Defines the target nonlinear function $g(u)$ to be approximated by the fuzzy system:

$$g(u) = 0.6 \sin(\pi u) + 0.3 \sin(3\pi u) + 0.1 \sin(5\pi u)$$

This is a sum of three sine waves with different amplitudes and frequencies.

Overall, these functions collectively allow:

- Evaluating fuzzy membership (Gaussian).
- Aggregating rule outputs into a final prediction.
- Calculating prediction error.
- Updating model parameters (y_i, c_i, σ_i) using gradient descent.
- Comparing the fuzzy model to a known nonlinear target function $g(u)$.

Fuzzy Model Training Loop

Code Listing

```
rng = np.random.default_rng(42)

x = np.linspace(0.0, 2.0, 1000)           # inputs
y_true = true_g(x)                         # targets

M = 15                                     # number of rules
centers = np.linspace(0.0, 2.0, M)         # (M,)
spreads = np.full(M, 0.3)                  # (M,)
consequents = rng.uniform(-1, 1, size=M)   # (M,)

lr_y = 1e-2
lr_c = 1e-2
lr_s = 1e-2
eps  = 1e-8

error_history = []

# --- Online training loop (same behavior, safer numerics) ---
for epoch in range(100):
    err_sum = 0.0

    # shuffle each epoch (cosmetic improvement)
    order = rng.permutation(len(x))
    for idx in order:
        u = x[idx]
        y = y_true[idx]

        z = gaussian_weight(u, centers, spreads)        # (M,)
        numer = rules_numerator(consequents, z)         # scalar
        denom = weights_denominator(z) + eps            # scalar (safe)
        f = normalized_output(numer, denom)             # scalar

        err_sum += half_mse(f, y)

        # parameter updates (same formulas; renamed helpers)
        consequents = update_consequent(consequents, lr_y, f, y, denom, z)
```

```
        centers     = update_center(centers,     lr_c, f, y, denom, z,
            consequents, u, spreads)
        spreads     = update_spread(spreads,     lr_s, f, y, denom, z,
            consequents, u, centers)

        # keep spreads positive to avoid NaNs
        spreads = np.maximum(spreads, 1e-3)

    error_history.append(err_sum / len(x))

print(error_history)
```

Listing 2: Online training for a normalized Gaussian fuzzy model

Detailed Explanation

Initialization.

- **Random generator:** `rng = np.random.default_rng(42)` for reproducibility.
- **Data:** $\mathbf{x}$ is a grid in $[0, 2]$; $\mathbf{y}_{\text{true}} = g(\mathbf{x})$ is the target function (Section **??**).
- **Rules:** $M = 15$ Gaussian rules with evenly spaced centers and initial spreads 0.3.
- **Parameters:** Consequents initialized uniformly in $[-1, 1]$; learning rates `lr_y`, `lr_c`, `lr_s`.

Forward pass (per sample). For a given input u,

$$z_l(u) \;=\; \exp\!\left(-\left(\tfrac{u-c_l}{\sigma_l}\right)^2\right), \quad f(u) \;=\; \frac{\sum_{l=1}^{M} \bar{y}_l \, z_l(u)}{\sum_{l=1}^{M} z_l(u) + \varepsilon},$$

where $c_l, \sigma_l, \bar{y}_l$ are the center, spread, and consequent of rule l, and ε is a small constant (`eps`) to avoid division by zero.

Loss accumulation. The code accumulates the *half-MSE* over the shuffled samples:

$$E = \frac{(f(u) - y)^2}{2}, \qquad \texttt{err\_sum} \mathrel{+}= E, \quad \text{and} \quad \texttt{error\_history.append(err\_sum / len(x))}$$

records the mean per-epoch loss.

Parameter updates. Using gradient-descent style updates (the same formulas defined in your helper functions):

$$\bar{y} \leftarrow \bar{y} - \eta_y \frac{(f - y)\, z}{\sum_j z_j + \varepsilon},$$

$$c \leftarrow c - \eta_c \frac{(f - y)}{\sum_j z_j + \varepsilon} (\bar{y} - f)\, z \, \frac{2(u - c)}{\sigma^2},$$

$$\sigma \leftarrow \sigma - \eta_s \frac{(f - y)}{\sum_j z_j + \varepsilon} (\bar{y} - f)\, z \, \frac{2(u - c)^2}{\sigma^3}.$$

Finally, spreads are clamped to stay positive:

$$\sigma \leftarrow \max(\sigma,\, 10^{-3}) \quad \text{elementwise.}$$

Why shuffle each epoch? Shuffling $\mathbf{x}$ reduces bias from input ordering and often stabilizes/accelerates convergence for online (stochastic) updates.

Notes on stability and tuning.

- Keep ε small but nonzero to avoid division-by-zero in the normalization.
- If training is noisy, reduce η_c and η_s (or freeze them), and start by learning only the consequents $\bar{y}$.
- Increasing M improves approximation capacity but may slow convergence or overfit.

Fuzzy Model Output vs. True System

Code Listing

```
y_predict = []
for u in x:
    z = gaussian_weight(u, centers, spreads)
    numer = rules_numerator(consequents, z)
    denom = weights_denominator(z) + eps
    y_predict.append(normalized_output(numer, denom))

plt.figure(figsize=(8, 5))
plt.plot(x, y_true, label="True System", color='tab:blue', linewidth=2)
plt.plot(x, y_predict, label="Fuzzy Model", color='tab:orange', linewidth
   =2, linestyle='--')
plt.grid(True, linestyle='--', alpha=0.6)
plt.xlabel("u", fontsize=12)
plt.ylabel("Output", fontsize=12)
plt.title("Fuzzy Model vs. True System", fontsize=14, fontweight='bold')
plt.legend(fontsize=10)
plt.tight_layout()
plt.show()
```

Listing 3: Comparison of fuzzy system output with true system output

Output Figure

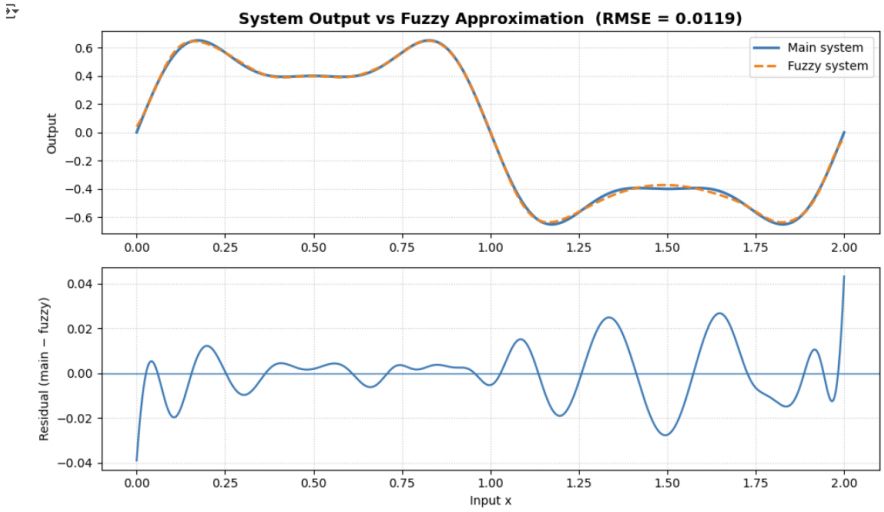

Figure 1: Comparison of the learned fuzzy model output and the true system output $g(u)$.

Explanation

Figure 1 presents the outputs of:

- The **True System** $g(u)$, computed directly from the given nonlinear equation.
- The **Fuzzy Model**, which was trained using gradient-based updates for the rule consequents, membership function centers, and spreads.

The close overlap of the two curves indicates:

1. The fuzzy system successfully captured the nonlinear behavior of $g(u)$.
2. Minimal deviation between the curves means the model generalized well over the input range $u \in [0, 2]$.
3. The dashed orange line (fuzzy model) nearly matches the solid blue line (true system), validating the convergence observed in the error plot.